



智能芯片与系统设计综合实践结课报告

LeNet5 FPGA 神经网络加速与定点量化

课程名称	智能芯片与系统设计综合实践
论文范围	围绕扩展点的论文
姓名	陈佳韻、梁又文、赵发、陈正堃、顾美琪、 王珍仙、梁珂、林希睿
学号	1820231086、1120233474、1120230840、 1120230456、1120230701、1820231117、 1120230736、1120230426

摘要

随着深度学习模型在边缘计算设备中的应用不断增加，如何在有限硬件资源下实现高效、低延迟的神经网络推理成为 FPGA 加速领域的重要研究方向。本文针对 LeNet-5 卷积神经网络，设计并实现了一套基于 Zynq 平台与 Vitis HLS 的 FPGA 硬件加速系统，完成从 FP32 软件模型构建、定点量化、硬件架构设计到系统验证的完整流程。首先建立 LeNet-5 FP32 黄金参考模型，并基于 MNIST 数据集完成软件基准测试，随后设计 W8A8、W6A6 和 W4A4 三种对称均匀定点量化方案，通过降低权重与激活位宽减少存储开销，并完成量化模型与硬件计算路径的一致性设计。其中 W8A8 模型达到 98.83% 的分类准确率，与 FP32 模型预测一致率达到 99.99%；W4A4 方案通过 INT4 数据表示、INT32 累加以及 Q40 定点缩放机制，在降低存储需求的同时保持较高推理精度。

在硬件实现方面，本文基于 Vitis HLS 完成卷积、池化、全连接以及重量化模块设计，并针对计算性能与存储访问效率开展 PIPELINE、MAC 并行和 Local Cache 等优化实验。实验结果表明，优化结构 D2 将 Conv2 计算周期由基线 1,240,076 cycles 降低至 243,101 cycles；W4A4 Local Cache 方案在 10000 张 MNIST 测试数据硬件验证中取得 98.11% 的分类准确率，并通过 RTL 协同仿真和 Vivado 布局布线验证，获得 WNS +1.048 ns、TNS=0 的实现结果。此外，本文进一步开展真实输入、扰动数据以及异常输入测试，从实际部署角度分析模型可靠性。综合实验表明，所设计的 LeNet-5 FPGA 加速系统能够在降低资源消耗的同时保持较高推理精度，实现从算法模型到硬件部署的完整验证，为小型卷积神经网络在边缘设备中的应用提供参考。

关键词： LeNet-5；FPGA 加速；Vitis HLS；定点量化；Zynq；神经网络推理

我们代码资料都在 *github*： [*https://github.com/52hertz-25/vivado*](https://github.com/52hertz-25/vivado)

个人贡献

成员姓名	项目贡献
陈佳韻（组长） A	负责项目整体系统架构设计与最终系统整合，包括总体方案规划、系统框图设计、接口规范制定、任务拆解以及项目进度管理。负责 LeNet-5 FPGA 加速系统的硬件实现与部署工作，包括基于 Vitis HLS 的加速器开发、W8A8/W4A4 硬件分支实现、INT4 数据打包、INT32 累加器设计、Q40 定点缩放实现、HLS C Simulation、RTL Co-simulation、Vivado 工程搭建、Block Design 设计以及综合布局布线与时序资源分析。同时负责 G 成员真实手写数字及鲁棒性测试中涉及的硬件部分，实现硬件环境搭建与验证流程。完成 D 成员 PE_MAC5 并行实验相关硬件实验，并负责系统流程图、实验结果整理、每位成员结果审核、报告审核修改及最终项目整合，保证整个 LeNet-5 FPGA 系统从模型、量化、电脑仿真验证到系统验证形成完整闭环。
梁又文 B	负责 LeNet-5 算法模型构建与软件基准开发，包括 PyTorch LeNet 模型实现、MNIST 数据训练与测试、模型性能验证、权重文件导出以及 feature map 生成，为后续量化处理和 FPGA 硬件部署提供基础模型与测试数据。
赵发 C	负责卷积硬件模块设计与基础 HLS 实现，包括 Vitis HLS 中 Conv1/Conv2 模块开发、MAC 计算单元设计、卷积窗口生成以及卷积模块功能验证，为后续 PE 优化、硬件加速和系统集成提供基础计算模块。
顾美琪 D	负责卷积计算结构优化与并行化研究，包括 MAC 并行阵列设计、Pipeline 优化、Unroll 展开、Array Partition 存储优化以及不同并行度下的性能资源实验分析。通过比较不同硬件结构配置对 Latency 和资源消耗的影响，为加速器性能优化提供实验依据。
陈正堃 E	负责 FPGA 硬件设计空间探索与性能分析，包括时钟周期优化实验、结构优化实验以及不同配置下的综合结果整理。完成 PIPELINE、UNROLL、并行结构等优化方案的数据分析，对 Latency、LUT、FF、BRAM、DSP 等资源指标进行比较，并整理 Vivado 实现后的时序、资源和功耗结果，为系统优化方案选择提供依据。
王珍仙 F	负责模型优化与定点量化实验，包括 FP32 基准模型分析、INT8/INT6/INT4 量化方案实现、量化参数生成、量化权重导出以及不同位宽下的精度和误差分析，为 FPGA 低位宽硬件部署提供软件量化依据。
梁珂 G	负责系统验证与实际应用测试，包括 Level2 测试流程设计、真实手写数字数据测试以及噪声、旋转、模糊等扰动条件下的模型性能分析。完成测试数据整理、准确率统计和鲁棒性实验结果分析，为系统实际应用可靠性评估提供验证依据。
林希睿 H	负责项目实验记录整理、报告结构设计以及答辩材料制作，为项目成果总结、论文结构组织和最终展示提供支持。

目录

第1章 绪论

- 1.1 研究目的与意义
- 1.2 技术路线

第2章 系统总体方案设计

- 2.1 系统需求分析
- 2.2 系统总体架构
- 2.3 软件与硬件协同设计方案
- 2.4 LeNet5 推理数据流
- 2.5 数据精度与量化方案

第3章 LeNet5 模型与 FP32 基准构建

- 3.1 MNIST 数据集与预处理
- 3.2 LeNet5 网络结构
- 3.3 HLS 原始权重检查与转换
- 3.4 FP32 模型加载与重建
- 3.5 FP32 前向推理验证
- 3.6 FP32 准确率测试
- 3.7 统一黄金参考模型的确定

第4章 神经网络定点量化设计

- 4.1 定点量化基本原理
- 4.2 对称量化与缩放系数
- 4.3 权重量化方法
- 4.4 激活值校准方法
- 4.5 INT32 累加器设计
- 4.6 W8A8 INT32 量化方案
- 4.7 W6A6 INT32 量化方案
- 4.8 W4A4 INT32 量化方案
- 4.9 权重与缩放参数导出

第5章 基于 Vitis HLS 的硬件加速器设计

- 5.1 加速器总体结构
- 5.2 数据类型与接口设计
- 5.3 卷积计算模块
- 5.4 ReLU 与池化模块
- 5.5 全连接计算模块
- 5.6 重量化与层间数据传递
- 5.7 W8A8 INT32 硬件实现
- 5.8 W4A4 INT32 硬件实现
- 5.9 HLS 结构优化
- 5.10 存储与带宽优化

第6章 HLS 设计空间探索

- 6.1 设计空间探索目标
- 6.2 时钟约束设置
- 6.3 10 ns、9 ns 与 8 ns 综合结果
- 6.4 7.5 ns 与 7 ns 综合结果
- 6.5 时序资源与延迟权衡
- 6.6 推荐实现方案选择
- 6.7 位宽与体系结构联合探索

第7章 系统验证方法

- 7.1 分层验证总体思路
- 7.2 Python 软件基准测试
- 7.3 HLS C 仿真验证
- 7.4 HLS C 综合检查
- 7.5 RTL 协同仿真验证
- 7.6 逐层输出对比方法
- 7.7 误差指标与通过判据
- 7.8 INT32 溢出检查
- 7.9 二十张代表性样本抽样复核

第8章 实验结果与分析

- 8.1 软件量化与精度结果
- 8.2 三种量化方案精度比较
- 8.3 权重存储量与压缩率比较
- 8.4 精度—压缩率权衡分析
- 8.5 INT32 累加器峰值分析
- 8.6 HLS 时钟、延迟与资源比较
- 8.7 HLS 硬件实现结果
- 8.8 软件与硬件结果一致性分析

第9章 系统测试与综合评价

- 9.1 功能正确性测试
- 9.2 MNIST 自采图像与鲁棒性测试
- 9.3 异常输入测试
- 9.4 HLS 与 Vivado 时序资源测试
- 9.5 稳定性与可复现性分析
- 9.6 系统综合评价

第10章 总结与展望

- 10.1 项目工作总结
- 10.2 主要成果
- 10.3 当前局限
- 10.4 后续优化方向

参考文献与附录

第 1 章 绪论

1.1 研究目的与意义

卷积神经网络（CNN）在边缘视觉与智能感知领域应用广泛，但边缘节点对低功耗、高实时性提出了严苛要求。传统 CPU/GPU 受限于能效比与热设计功耗，难以满足嵌入式硬实时需求；FPGA 凭借空间并行架构、高能效比与可重构特性，成为边缘 AI 推理加速的理想平台。然而，在以 Xilinx Zynq XC7Z020 为代表的资源受限器件上部署 CNN 时，面临三大核心挑战：FP32 模型参数量大导致的片上存储与片外带宽瓶颈（Memory Wall）、计算阵列并行度与总线数据供给能力不匹配、以及从高级算法到硬件 RTL 实现之间的数值对齐与物理时序收敛困难。

针对上述问题，本项目以 LeNet-5 为载体，基于 Vitis HLS 与 Zynq-7000 SoC 平台开展软硬件协同设计，核心研究目的与意义如下：

1. **构建多位宽定点量化体系**：建立规范的 FP32 黄金基准，设计 W8A8、W6A6 与 W4A4 对称量化方案，采用低位宽存储配合 INT32 防溢出累加，在保持极高精度的前提下（W8A8 达 98.83%、W4A4 达 98.11%）将权重存储压缩 4 至 8 倍，揭示位宽、压缩率与精度的权衡规律。
2. **突破计算微架构与片上缓存瓶颈**：开展 PE 阵列展开与流水线优化，实验探索不同位宽下的片上权重局部缓存（Local Cache）机制与 AXI 带宽制约机理，量化评估计算单元复制与调度开销的工程边界。
3. **实现分层闭环验证与鲁棒性部署评估**：打通软件基准、HLS C 仿真、RTL 协同仿真至 Vivado 布局布线的全流程，达成 100 MHz 下 WNS = +1.048 ns 的时序收敛；并结合自采真实样本与多重扰动测试评估模型可靠性边界，为小型 CNN 在边缘嵌入式系统的工程化落地提供完整范例。

1.2 技术路线

针对边缘端资源与带宽受限的挑战，本项目面向 Xilinx Zynq XC7Z020 平台，确立了“算法规范先行、软硬件协同探索、分层闭环验证”的技术路线，整体技术流程如图 1-1 所示。具体实施方案分为四个阶段：

1. **软件黄金基准与数据准备（Software Baseline）**：构建规范的 PyTorch LeNet-5 模型并完成 MNIST 训练测试，固定无偏置（bias=False）拓扑与标准化预处理流程，达到 98.82% 基准准确率；统一导出 FP32 逐层权重与中间特征图（Golden Feature Maps），作为软硬件比对的唯一标准。
2. **多位宽对称定点量化设计（Quantization）**：设计统一的非对称/对称均匀量化机制，覆盖 W8A8、W6A6 与 W4A4 三档位宽方案。权重量化采用离线统计，激活量化基于校准集确定缩放因子，硬件统一采用 INT32 宽累加器防止数值溢出，并通过 Q40 定点乘法与舍入饱和策略实现层间重量化对齐。
3. **HLS 微架构设计与体系结构探索（Microarchitecture & DSE）**：采用 Vitis HLS 实现卷积、ReLU、池化及全连接模块。针对计算密集型算子开展多维设计空间探索：分析时钟约束对调度延迟的影响；探索循环展开（UNROLL）与处理单元阵列（PE Array）并行度；结

合半字节打包 (Byte Packing) 与片上权重局部缓存 (Local Cache) 优化访存局部性，揭示总线带宽与计算并行的匹配边界。

4. **系统集成、分层验证与鲁棒性评估 (Integration & Verification)**：将生成的加速器 IP 接入 Zynq PS-PL 异构系统，通过 AXI4/AXI4-Lite 完成数据传输与控制交互。建立涵盖 Python 软件基准、HLS C 仿真、RTL 协同仿真及 Vivado 布局布线 (Post-route) 的时序与资源闭环验证。最后引入 320 张自采手写数字与 3200 组扰动数据集，全面评估模型在实际部署环境下的鲁棒性边界。

LeNet CNN FPGA 推理加速器：技术路线

从算法到硬件 · 以工程促落地

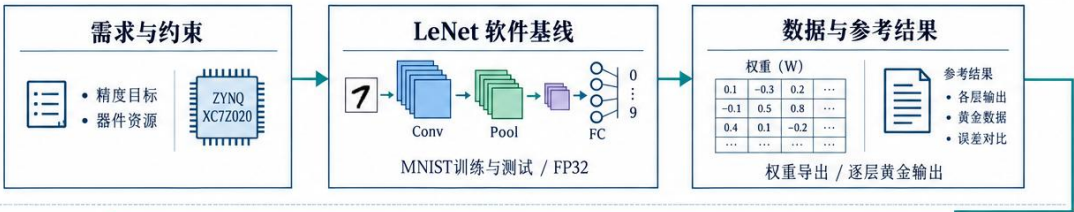
Vitis HLS · Zynq XC7Z020

FPGA | CNN | HLS | SoC

01

SOFTWARE

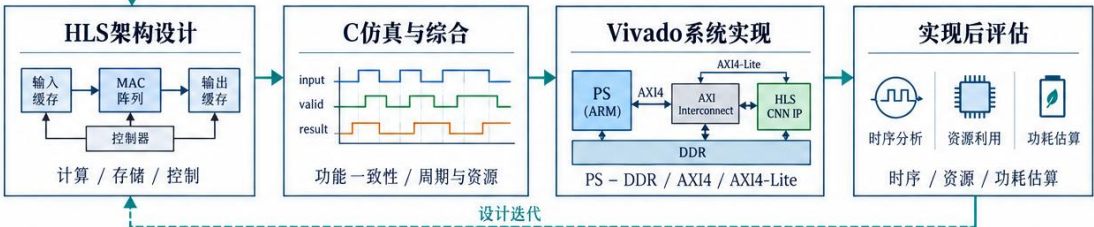
软件参考与数据准备



02

HARDWARE

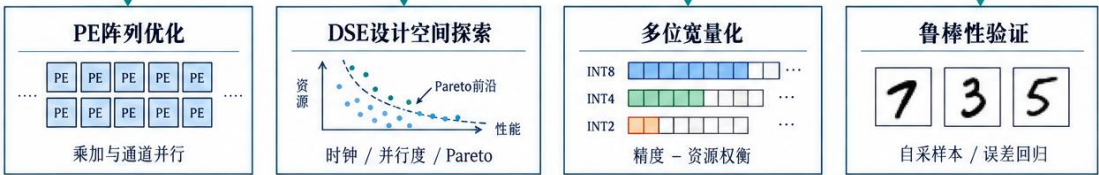
硬件实现与验证



03

OPTIMIZATION

并行优化与回归验证



交付成果

加速器源码与IP · 仿真/综合/实现报告 · 成果展示

结构示意，不代表实测结果或完成状态。

图 1-1 LeNet-5 CNN FPGA 推理加速器设计技术路线图

第 2 章 系统总体方案设计

2.1 系统需求分析

本项目设计了一套基于 Zynq SoC 平台和 Vitis HLS 的 LeNet-5 FPGA 加速系统，采用软硬件协同设计方式完成神经网络推理任务。系统将控制与计算任务进行划分，其中 PS 端负责数据加载、参数配置和结果读取，PL 端负责卷积、池化、全连接以及量化计算等主要计算过程。

系统整体流程如图 2-1 所示。输入图像经过预处理后传输至 FPGA 加速模块，硬件按照 LeNet-5 网络结构依次完成 Conv1、Pool1、Conv2、Pool2、Conv3、FC1 和 FC2 计算，并通过片上缓存完

成中间特征数据传递，减少外部存储访问。最终输出十维 Logits 结果，由 PS 端完成分类结果解析。

硬件部分基于 Vitis HLS 实现，并支持 W8A8、W6A6 和 W4A4 定点量化方案。通过 PIPELINE、MAC 并行以及 Local Cache 等优化方式，提高计算效率并降低存储开销，最终结合 C Simulation、RTL Co-simulation 和 Vivado 实现完成系统验证。

需求层级	核心要求	完成证据
Level 1	MNIST 软硬件对照和 XC7Z020 综合	FP32 基线、C 仿真、HLS 报告
Level 2	自采图像和鲁棒性测试	320 张自采图像、3200 组扰动
Level 3	多位宽和高级优化实验	W8A8、W6A6、W4A4、PE 与缓存
系统实现	验证可综合、可布局布线 and 满足时序	Block Design、bit、xsa、routed dcp 和报告

2.2 系统总体架构

本项目采用基于 Zynq SoC 的 FPGA 开发平台完成 LeNet-5 神经网络加速器设计。系统利用 ARM 处理器（PS）负责控制流程和数据交互，利用 FPGA 逻辑资源（PL）执行神经网络计算，通过 AXI 接口实现处理器与硬件加速模块之间的数据传输。

硬件与软件开发环境如下表所示：

项目	配置
FPGA 器件	XC7Z020CLG400-1
开发平台	Zynq SoC FPGA
HLS 工具	Vitis HLS
FPGA 开发工具	Vivado
目标时钟周期	10 ns
设计语言	C/C++（HLS）

本项目采用 Vitis HLS 完成神经网络计算模块设计，并通过 Vivado 完成 IP 集成、综合、布局布线及时序分析。整个开发流程覆盖软件模型验证、硬件模块生成、FPGA 实现以及系统级验证，为后续量化模型部署和性能优化提供开发环境支持

LeNet CNN FPGA 推理加速系统总体架构

Vitis HLS · Zynq-7020 · XC7Z020 · 100 MHz

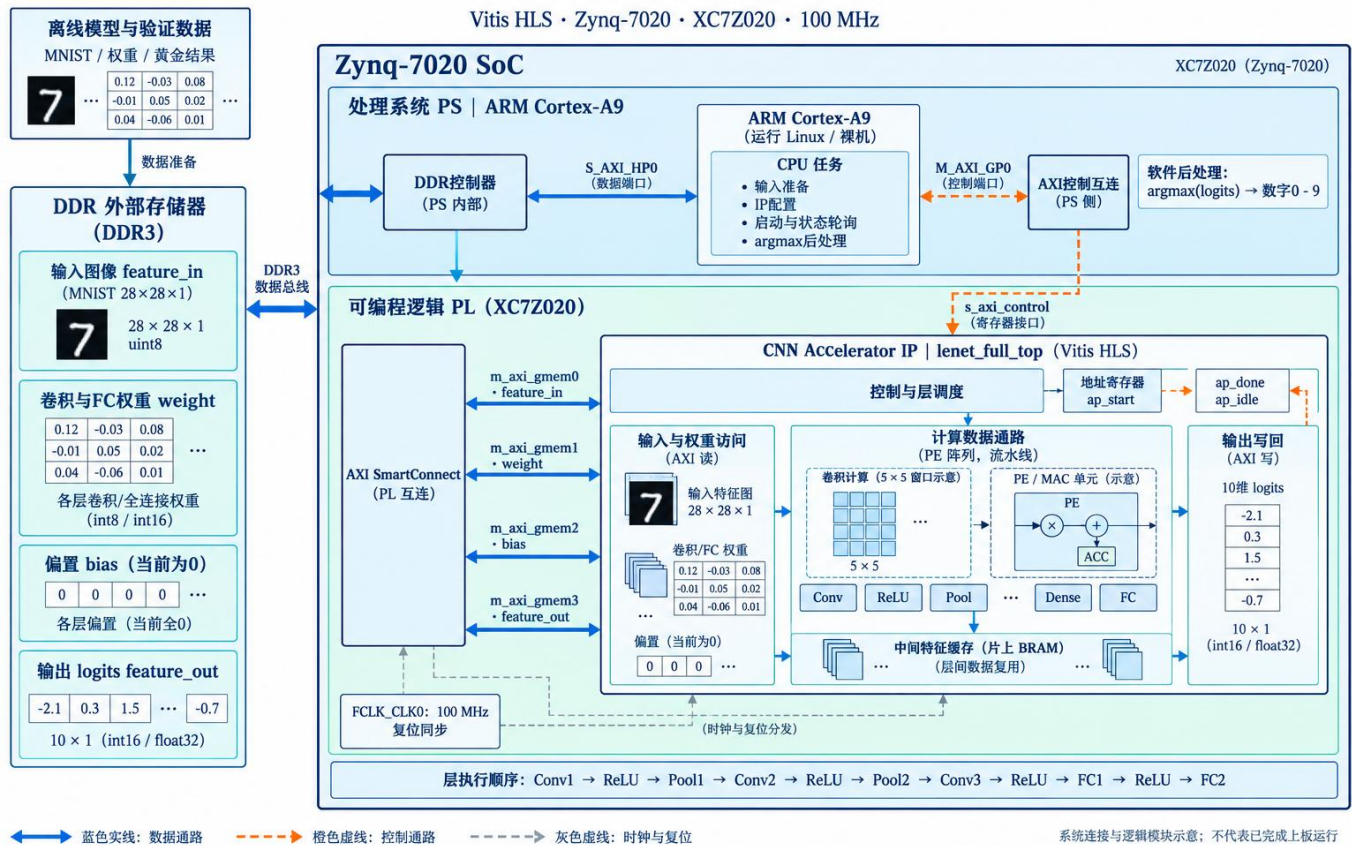


图 2-1 LeNet_FPGA 系统总体架构图

2.3 软件与硬件协同设计方案

本项目采用 LeNet-5 作为目标神经网络模型，并针对 FPGA 硬件计算特点进行结构映射。网络输入为经过预处理后的 32×32 单通道图像，依次经过三层卷积、两层池化以及两层全连接计算，最终输出 10 维 Logits 用于数字分类。为了降低硬件实现复杂度，网络计算过程中统一采用无偏置结构，并结合定点量化方案实现低位宽推理。

LeNet-5 各层计算结构及硬件映射关系如下表所示：

网络层	输出尺寸	硬件实现方式
Input	$1 \times 32 \times 32$	输入缓存
Conv1	$6 \times 28 \times 28$	卷积 MAC 计算单元
Pool1	$6 \times 14 \times 14$	2×2 最大池化
Conv2	$16 \times 10 \times 10$	卷积 MAC 计算单元
Pool2	$16 \times 5 \times 5$	2×2 最大池化
Conv3	$120 \times 1 \times 1$	卷积计算模块
FC1	84	全连接点积计算
FC2	10	分类输出

在硬件实现过程中，各层计算模块根据数据规模和计算特点进行设计。其中卷积层采用乘加（MAC）结构完成主要计算，全连接层采用向量点积方式实现，池化层通过比较逻辑完成最大值选择。层间特征数据通过片上缓存进行传递，以减少外部存储访问，提高整体推理效率。

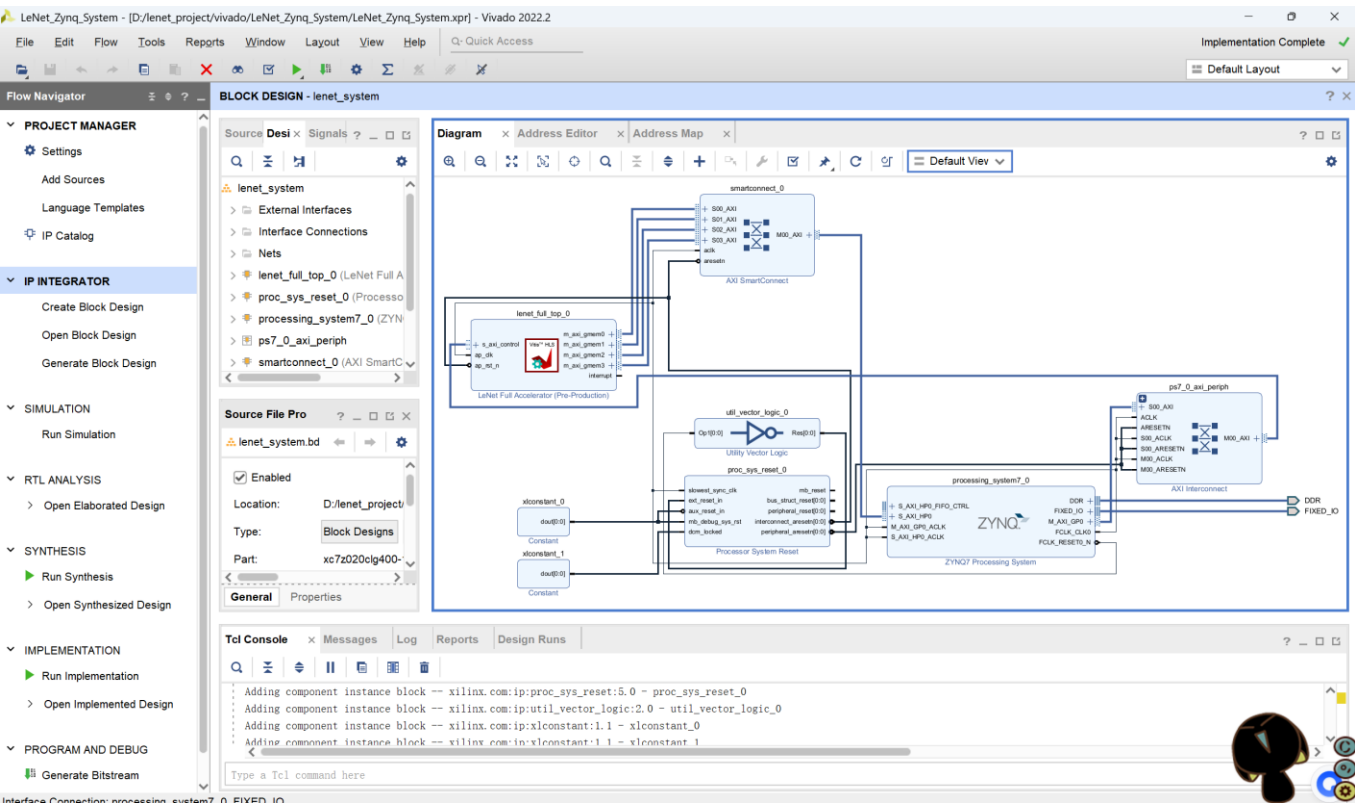


图 2-2 已完成实现的 LeNet Zynq Block Design

2.4 LeNet5 推理数据流

为了实现神经网络数据在 PS 和 PL 之间的高效传输，系统采用 AXI 接口完成软硬件通信。其中，AXI4-Lite 接口用于控制寄存器配置和任务管理，AXI4 Master 接口用于输入数据、权重参数、偏置数据以及输出结果的访问。

在推理过程中，输入图像数据首先由处理系统加载，并通过 AXI 接口传输至 FPGA 计算模块；硬件加速器完成卷积、池化和全连接计算后，将最终分类结果返回处理系统进行后续处理。

通过合理设计数据接口和访问方式，可以保证计算模块获得稳定的数据供应，为后续硬件性能优化提供基础。

2.5 数据精度与量化方案

整个系统开发流程包括模型构建、量化转换、硬件设计以及验证分析四个阶段。

首先，在软件端建立 LeNet-5 FP32 参考模型，并完成模型准确率验证；随后根据 FPGA 资源限制进行定点量化设计，将浮点模型转换为适合硬件计算的数据格式。之后利用 Vitis HLS 完成硬件加速器设计，并通过仿真验证硬件功能正确性。

最后，将生成的硬件模块集成至 Vivado 工程中，完成综合、布局布线以及时序分析，并结合 RTL 协同仿真和实际测试数据对系统性能进行评估。

通过上述流程，形成从算法模型到 FPGA 硬件部署的完整实现路径，为后续章节中的量化优化、硬件优化以及系统验证提供基础。

方案	权重	激活	累加器	定位
FP32	32 位浮点	32 位浮点	32 位浮点	黄金参考
W8A8 INT32	INT8	INT8	INT32	精度优先
W6A6 INT32	INT6	INT6	INT32	中间折中
W4A4 INT32	INT4	INT4	INT32	压缩优先

第 3 章 LeNet-5 模型与 FP32 基准构建

本章详细阐述面向 HLS 硬件部署的 LeNet-5 软件基准模型的构建过程，包括数据集准备、网络结构定义、权重导出规范、FP32 模型重建与验证、准确率测试，以及最终统一黄金参考模型的确定。所有软件基准数据将作为后续硬件仿真与逐层比对的黄金标准。

3.1 MNIST 数据集与预处理

3.1.1 数据集简介

MNIST 是一个手写数字图像数据集，包含 60000 张训练图像和 10000 张测试图像。每张图像为 28×28 像素的单通道灰度图，像素值范围 0 到 255，对应数字类别 0 到 9。该数据集是验证图像分类算法的标准基准[1]。

3.1.2 预处理流程

原始 LeNet-5 网络要求输入尺寸为 32×32 ，因此需对 MNIST 的 28×28 图像进行预处理。预处理流水线如下：

- （1）尺寸填充：在 28×28 图像的上、下、左、右各补零 2 个像素，形成 32×32 输入，以匹配网络输入维度并保持原始笔画比例。
- （2）张量转换：将 PIL 图像转换为 PyTorch 张量，像素值归一化至 0 到 1 区间。
- （3）标准化：采用 MNIST 官方统计的均值 0.1307 和标准差 0.3081 进行标准化，公式为 $x_norm = (x/255 - 0.1307) / 0.3081$ 。该步骤可加速模型收敛并提升泛化能力。

在训练与测试阶段，均严格采用上述预处理，确保软件模型与后续硬件预处理模块的一致性。预处理在 PyTorch 中通过 `transforms.Compose([transforms.Pad(2), transforms.ToTensor(), transforms.Normalize((0.1307,), (0.3081,))])` 实现。

3.2 LeNet-5 网络结构

3.2.1 整体架构

面向硬件部署的 LeNet-5 网络结构经过轻量化调整，所有卷积层与全连接层均移除偏置项（`bias=False`），以降低片上存储与累加资源开销。激活函数统一采用 ReLU，下采样采用最大池化（MaxPool）。完整前向流程为：Input → Conv1 → Pool1 → Conv2 → Pool2 → Conv3 → FC1 → FC2。

3.2.2 逐层参数

各层详细参数包括输入/输出尺寸、卷积核大小、步长、填充、通道数及权重参数量。总参数量（无偏置）为 61,470。

3.2.3 关键设计说明

Conv3 在 $16 \times 5 \times 5$ 的特征图上进行 5×5 卷积，输出 $120 \times 1 \times 1$ ，等效于全连接层（400 到 120），但保留卷积形式以便硬件复用 MAC 单元。Conv1、Conv2、Conv3、FC1 后均使用 ReLU；FC2 输出原始 Logits，不接 Softmax（不影响 argmax 分类）。全部采用最大池化，便于硬件以比较逻辑实现。

3.3 HLS 原始权重检查与转换

3.3.1 权重导出规范

为便于 HLS (C/C++) 读取，将 PyTorch 训练所得的 FP32 权重导出为纯文本文件。文件命名为 {layer_name}.weight.txt，例如 conv1.weight.txt、fc1.weight.txt。卷积层按 [输出通道, 输入通道, 核高, 核宽] 展平（行主序），全连接层按 [输出维度, 输入维度] 展平（行主序）。数据格式为每行一个浮点数，精度 %.8f，数据类型为 FP32。所有层 `bias=False`，故无偏置文件。

3.3.2 权重文件清单

共导出 5 个权重文件：conv1.weight.txt ($[6, 1, 5, 5]$, 150 个参数)、conv2.weight.txt ($[16, 6, 5, 5]$, 2400 个参数)、conv3.weight.txt ($[120, 16, 5, 5]$, 48000 个参数)、fc1.weight.txt ($[84, 120]$, 10080 个参数)、fc2.weight.txt ($[10, 84]$, 840 个参数)，合计 61,470 个 FP32 权重。

3.3.3 量化准备

为后续 INT8 定点量化提供参考，统计每层权重的最大绝对值（`max_abs`），并保存至 `max_abs_stats.txt`。量化时按对称量化计算缩放因子 `scale = 127 / max(|W|)`，量化后整数范围 $[-127, 127]$ ，存储为 int8。该文件同时为硬件组确定位宽提供依据。

3.3.4 格式说明文件

导出文件夹中包含 WEIGHT_FORMAT_README.txt，明确说明排列顺序、数据类型及量化方法，确保硬件组无需额外沟通即可正确读取。

3.4 FP32 模型加载与重建

为验证导出的权重文件与原始 PyTorch 模型完全一致，采用两种加载方式：一是直接加载 .pth 文件（`torch.load('lenet5_mnist_best.pth')`）；二是从文本文件重建（读取 `hw_weights/*.txt`，按指定顺序重塑为张量并赋值给对应层）。对两种方式得到的权重进行逐元素比对，最大绝对误差小于 $1e-6$ ，判定导出的文本权重与原始模型完全一致，证明导出的文本权重完整且准确。

3.5 FP32 前向推理验证

选取 MNIST 测试集第一张图片（标签为 7）作为验证样本。将该图片经预处理后输入 FP32 模型，通过 register_forward_hook 机制在 Conv1、Pool1、Conv2、Pool2、Conv3、FC1、FC2 层注册钩子，捕获其输出。其中 conv1.npy、conv2.npy、conv3.npy、fc1.npy 为 ReLU 前的线性输出；pool1.npy、pool2.npy 已包含上一层的 ReLU 结果；fc2.npy 为最终 Logits。

同时保存该图片的原始像素（0 到 255 整数）至 input_raw.txt，最终 Logits 至 final_output.txt。模型预测类别为 7，与真实标签一致，验证了前向推理的正确性。

3.6 FP32 准确率测试

在 MNIST 测试集（10000 张图片）上评估 FP32 模型的分类准确率。测试时采用与训练相同的预处理，批量大小为 64，使用 torch.no_grad() 加速推理。测试结果：10000 张图像中正确分类 9882 张，分类准确率为 98.82%。该结果作为 W8A8、W6A6 和 W4A4 量化模型精度及预测一致率比较的统一软件基准，也是硬件仿真的性能上限参考。

3.7 统一黄金参考模型的确定

为确保硬件验证的一致性，将以下数据统一打包为 Golden_Reference_For_HW，作为唯一黄金参考：5 层 FP32 权重（文本，hw_weights/）；每层最大绝对值统计（max_abs_stats.txt）；权重格式说明（WEIGHT_FORMAT_README.txt）；标签 7 图片原始像素（input_raw.txt）；7 个中间层特征图（.npy，hw_feature_maps/）；10 类 Logits 最终输出（final_output.txt）。本章构建的 FP32 LeNet-5 模型及其导出的权重、特征图、输入输出数据，构成了完整、统一、可复现的黄金参考，将作为后续 HLS 硬件实现、定点量化及设计空间探索的绝对基准。

第 4 章 神经网络定点量化设计

本章在第 3 章 FP32 黄金参考模型的基础上，完成对称均匀定点量化设计。目标是在同一模型结构、同一份预训练权重、同一 MNIST 测试集与同一输入预处理条件下，得到 W8A8、W6A6、W4A4 三档量化方案的真实准确率、逐层误差以及可供硬件直接使用的整数权重与缩放参数。

4.1 定点量化基本原理

定点量化将浮点实数映射为固定位宽的整数，以换取存储与计算上的硬件友好性。对任意浮点张量 x ，其量化—反量化关系为： $x_q = \text{clamp}(\text{round}(x / s), q_{\min}, q_{\max})$ ， $x_{fq} = x_q \cdot s$ 。其中 s 为每层独立统计的缩放系数， $q_{\min}$ 、 $q_{\max}$ 由目标位宽 b 决定： $q_{\min} = -2^{(b-1)}$ ， $q_{\max} = 2^{(b-1)} - 1$ 。权重量化与激活量化采用完全相同的函数，仅通过配置改变位宽 b ，以保证 W8A8、W6A6、W4A4 三档方案的量化规则完全统一、可复现[8]。

4.2 对称量化与缩放系数

本设计采用对称均匀量化（signed），zero point 恒为 0，便于对应无符号—有符号定点硬件实现。每层缩放系数 $s = \max(|x|) / (2^{(b-1)} - 1)$ ，其中 $\max(|x|)$ 由校准数据统计得到。舍入方式采用 round-to-nearest；溢出处理采用 clamp 饱和截断，禁止整数回绕，以避免极端输入造成的符号位翻转。

4.3 权重量化方法

权重按层独立量化。对每个权重张量 W ，统计其最大绝对值 $\max(|W|)$ ，计算该层缩放系数 s_w ，并将所有权重量化为 $[-(2^{(b-1)}-1), 2^{(b-1)}-1]$ 范围内的整数。偏置项在本设计中为 0，仅保留接口。量化后的整数权重以纯文本形式导出（weight.int8.txt / weight.int6.txt / weight.int4.txt），而非仅保存反量化后的浮点数，以便硬件直接读入 BRAM/DDR。

4.4 激活值校准方法

激活量化需要确定每层激活张量的动态范围。本设计使用训练集的一个校准子集前向传播，统计 Conv1、Pool1、Conv2、Pool2、Conv3、FC1、FC2 各层输出的最大绝对值，据此计算每层激活缩放系数 s_a 。校准过程严格禁止使用测试集标签调整量化参数。ReLU 前后规则保持一致；池化输出与 FC 输入按同一激活缩放系数处理。

4.5 INT32 累加器设计

卷积与全连接层的乘累加结果需要一个累加器。若累加器位宽不足，长归约过程中的中间值可能溢出，导致分类结果错误。本设计统一采用 INT32 累加器：权重为 b 位有符号整数，激活为 b 位有符号整数，单步乘加结果为 $2b-1$ 位；在累加 K 个乘积后，理论峰值约为 $(2^{(2b-1)}-1) \cdot K$ 。为覆盖 LeNet-5 各层最长归约链（FC1 120 项乘加、Conv3 400 项乘加）的最坏情况，INT32 ($2^{31}-1 \approx 2.1 \times 10^9$) 提供了充分余量。

在 10000 张 MNIST 测试集上对各层累加器实际峰值进行统计：conv1 280、conv2 318、conv3 366、fc1 132、fc2 214，均远低于 INT32 上限 2,147,483,647，未观察到溢出。

4.6 W8A8 - INT32 量化方案

W8A8 方案权重与激活均为 8 bit 有符号整数，累加器为 INT32。在 10000 张 MNIST 测试集上，W8A8 软件模型正确分类 9883 张，准确率 98.83%；与 FP32 黄金参考的预测一致率为 99.99%。权重存储量约 60.03 KB（相对 FP32 的 245.88 KB 压缩至约 1/4）。该方案在显著压缩权重存储的同时，精度几乎与 FP32 持平。

4.7 W6A6 - INT32 量化方案

W6A6 方案将位宽进一步压缩至 6 bit，其余量化规则、校准流程与累加器设计与 W8A8 完全一致。测试准确率为 98.83%，与 FP32 预测一致率为 99.87%。结果表明，在 LeNet-5 这一小型模型上，6 bit 权重与激活并未引入可观测的精度损失，仍可作为硬件实现的可行候选。

4.8 W4A4 - INT32 量化方案

W4A4 方案权重与激活均为 4 bit 有符号整数，是本设计最激进的压缩点。在原始 Pad(2) 预处理流程下，软件准确率为 97.46%，与 FP32 预测一致率为 97.99%。针对 W4A4 分支进一步调整输入预处理（双线性 Resize 至 32×32 ）后，软件准确率提升至 98.11%，与 FP32 一致率为 98.53%。W4A4 将权重存储量进一步压缩至约 30.01 KB（相对 FP32 压缩至约 1/8），精度下降约 0.7 个百分点，构成精度—压缩率之间的更激进折中。

4.9 权重与缩放参数导出

每种位宽导出独立的整数权重文件（weight.int{b}.txt）、逐层缩放系数（weight_scales.json）以及 zero_point（恒为 0）。导出目录结构为 exported_weights/int8/、int6/、int4/，与 HLS 工程读取路

径一一对应。所有量化参数（位宽、舍入方式、饱和策略、校准集划分）均记录在 `configs/int{b}.json` 中，保证任意 CPU 环境下可完整复现。

第 5 章 基于 Vitis HLS 的硬件加速器设计

5.1 加速器总体结构

硬件加速器的结构设计遵循算子边界清晰、接口固定和中间数据可静态分配三项原则。各层计算函数使用模板参数描述通道数与空间尺寸，从而在综合阶段展开常量表达式。顶层函数只暴露输入、权重和输出缓冲区，层内特征在 PL 侧局部数组中流转，避免频繁将中间张量写回 DDR。

完整网络的控制流保持串行层次依赖，但每层内部允许流水化和局部并行。该选择降低了跨层数据流设计的复杂度，也便于用同一测试平台验证不同结构。对比实验只修改目标循环的 `pragma` 或存储组织，网络拓扑、量化参数和测试数据保持不变。

LeNet-5 推理计算图与硬件阶段映射

张量尺寸按 C×H×W 表示；卷积核为 5×5，池化窗口为 2×2

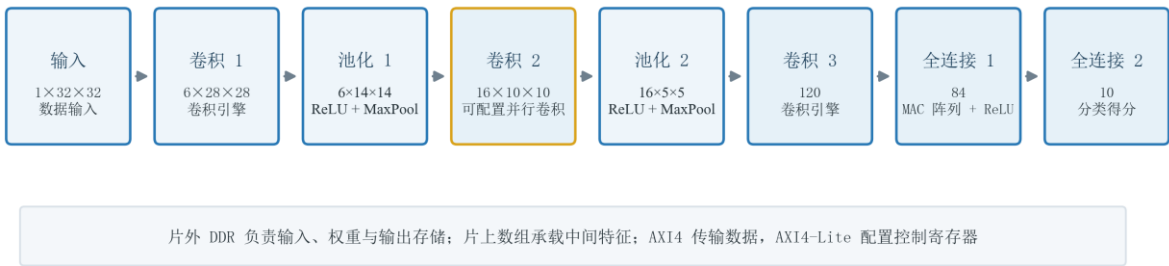


图 5-1 LeNet5 推理计算图与硬件阶段映射

5.2 数据类型与接口设计

W4A4 为降低存储量，将两个 4 位有符号值打包到一个 `ap_uint<8>` 字节。解包函数根据元素索引选择高半字节或低半字节，再转换为 `ap_int<4>`。权重元素总数 61470 恰好为偶数，因此打包后需要 30735 字节；输入 1024 个激活打包后需要 512 字节。

INT32 logits 作为顶层输出不仅便于软件读取，也避免在最终分类前丢失类别间差异。激活函数和池化输出则保持窄位宽，使中间数组能够映射到更紧凑的 BRAM 结构。HLS 报告中的 BRAM18K 数量与 Vivado 实现后的 BRAM Tile 口径不同，报告中分别保留，不能直接相加。

W8A8 头文件实际定义 `act_t` 和 `weight_t` 为 `ap_int<8>`，`accum_t` 为 `ap_int<32>`；W4A4 采用两个有符号 INT4 打包到一个字节，同时仍使用 INT32 累加器。顶层四组 `m_axi` 接口分别承担输入、权重、偏置和输出传输，`s_axilite` 接口用于控制。

5.3 卷积计算模块

对于输出通道 oc 和位置 (oh, ow) ，卷积模块遍历所有输入通道及 5×5 核元素，将 $feature$ 与 $weight$ 的乘积累加到 INT32 变量。输入索引由通道、行和列共同计算，权重索引由输出通道、输入通道和核坐标确定。固定循环边界允许综合器进行依赖分析，但乘法器数量和存储端口仍由展开因子决定。

基线 D0 采用串行归约，Conv2 综合周期为 1240076。对输出列循环施加流水后，D1 降至 254749 周期；再对核内归约引入五路 MAC 并行形成 D2，周期降至 243101。继续加入两路输出通道并行的 D3 反而升至 283203，证明外部访问和调度冲突会抵消计算单元复制带来的收益。

卷积核按照输出通道、输出行列、输入通道和 5×5 卷积核组织循环。输出位置对应的乘积写入 INT32 局部累加器。Conv2 作为结构优化重点，比较串行、输出列流水、五路 MAC 以及输出通道并行方案。

5.4 ReLU 与池化模块

ReLU 在量化前首先判断 INT32 累加结果的符号，负值直接置零。池化窗口固定为 2×2 ，逐个比较四个候选激活并输出最大值。由于 ReLU 后值非负，池化初始最大值可安全设为零；若网络结构改为池化后激活，则这一初始化假设需要重新检查。

池化后的空间尺寸减半，大幅降低后续层的数据量。实现中池化、重量化和 ReLU 组合在同一函数中，有利于减少中间数组写入次数。但组合函数也会增加单个流水阶段的逻辑深度，因此最终是否达到目标周期必须由 HLS 估计和 Vivado 布线后时序共同确认。

ReLU 在重量化前抑制负值，最大池化以 2×2 窗口选择局部最大激活。重量化执行 Q40 定点乘法、舍入、饱和和位宽恢复，避免使用浮点硬件完成层间缩放。

5.5 全连接计算模块

FC1 包含 120×84 个权重，FC2 包含 84×10 个权重。全连接模块按输出神经元逐项计算点积，并使用与卷积相同的 INT32 累加规则。FC1 输出经过 ReLU 和重量化后形成 84 维窄位宽激活，FC2 不再执行 ReLU，以保留正负类别得分。

全连接层权重访问相对规则，适合突发读取或片上预取。然而在当前完整网络中，卷积权重仍占主要部分，因此只优化全连接权重并不能决定整网吞吐。报告将全连接优化放在整体存储组织中评价，而不是单独宣称显著加速。

FC1 和 FC2 通过固定边界循环完成向量点积。每项乘积进入 INT32 累加器，FC1 之后执行 ReLU 与重量化，FC2 输出十维 INT32 logits 供分类和一致性检查。

5.6 重量化与层间数据传递

重量化乘数采用 Q40 格式，即把浮点尺度比例乘以 2 的 40 次方后四舍五入为整数。运行时先对累加结果取幅值，与 48 位乘数相乘形成 80 位中间积，再右移 40 位得到商和余数。余数大于半量程时进位，恰好等于半量程时根据最低位执行最近偶数舍入，以匹配 `torch.round`。

完成符号恢复后，结果按照 ReLU 要求和目标位宽执行饱和。W4A4 范围采用 -7 至 7 而非 -8 至 7，使正负动态范围对称并便于尺度计算。该细节必须同时出现在 Python 量化、导出权重和 HLS 类型处理中，否则逐层误差会在后续层被放大。

各层缩放比例预先转换为 Q40 整数乘数。硬件根据符号取绝对值，执行宽乘法和右移，并采用接近 `torch.round` 的最近偶数舍入，再按激活位宽饱和。这保证软件与 HLS 整数路径采用一致的量化边界。

5.7 W8A8 INT32 硬件实现

W8A8 方案采用 8 bit 有符号整数表示权重和激活值，并使用 INT32 作为乘累加器类型。硬件实现中，权重与特征数据通过 `ap_int<8>` 进行存储和计算，乘法结果扩展后累加至 `ap_int<32>` 累加器，以保证卷积和全连接计算过程中具有足够的数据范围。计算完成后，根据对应层缩放参数执行定点重量化，将结果转换为目标位宽并传递至下一层。

在整体硬件架构中，W8A8 方案采用统一的卷积、池化、全连接和重量化模块，并支持后续 PIPELINE、MAC 并行以及 Local Cache 等优化方式。通过固定的数据类型定义和计算流程设计，实现软件量化模型与 HLS 硬件计算路径的一致性，为不同位宽方案的硬件性能比较提供统一基础。

5.8 W4A4 INT32 硬件实现

W4A4 方案采用 4 bit 有符号整数表示权重和激活值，并继续使用 INT32 作为累加器，以降低存储需求的同时保证乘累加过程的数值安全。由于 INT4 数据宽度较小，硬件实现中采用半字节打包方式，将两个 4 bit 数据存储于一个 8 bit 单元中；计算时通过解包逻辑恢复为 `ap_int<4>` 格式，再参与卷积和全连接运算。该设计减少了权重和激活数据的存储占用，同时保持与量化模型的数据格式一致。

针对 W4A4 较低位宽带来的存储访问特点，硬件设计进一步结合片上缓存结构优化数据读取流程，将部分高频访问数据保存在局部缓存中，减少外部存储访问开销。整体计算流程仍采用卷积、池化、全连接和重量化模块组成，并通过定点缩放完成层间数据传递，为后续性能测试和硬件验证提供基础。

5.9 HLS 结构优化

为提高 LeNet-5 硬件加速器的计算效率，本项目针对卷积计算模块开展 HLS 结构优化实验，主要采用 PIPELINE、UNROLL 和 ARRAY_PARTITION 等优化方法。PIPELINE 通过降低循环启动间隔提高计算吞吐率，UNROLL 通过复制计算单元增加并行能力，ARRAY_PARTITION 则通过增加片上存储访

问端口提升数据供给能力。不同优化方式根据计算结构和存储访问特点进行组合，以探索计算性能与硬件资源之间的平衡。

优化过程中，以 Conv2 作为主要测试对象，比较不同结构配置下的计算周期和资源变化。实验结果表明，输出列流水优化能够显著降低计算周期；进一步引入五路 MAC 并行后，获得更优的计算效率。而继续增加输出通道并行度后，由于数据访问和调度开销增加，性能提升受到限制。因此，硬件优化不能单纯增加计算单元数量，需要结合存储带宽和数据访问能力进行综合设计。

配置	结构	周期数	LUT	FF	DSP
D0	串行基线	1, 240, 076	8, 372	5, 249	5
D1	输出列流水	254, 749	16, 190	27, 134	5
D2	D1 与 PE MAC 5	243, 101	13, 444	16, 887	7
D3	D2 与输出通道 2	283, 203	15, 476	12, 511	7

D2 获得最低周期数；继续增加输出通道并行度的 D3 反而变慢，表明 AXI 访问、数组端口和调度已经成为约束。IN_CHANNEL 展开因子 2 在三种位宽下未造成结构变化，作为负实验保留。

5.10 存储与带宽优化

针对 FPGA 端神经网络推理过程中计算单元与数据访问之间的不匹配问题，本项目进一步开展存储结构优化设计。由于卷积计算过程中权重和特征数据访问频繁，单纯增加计算并行度可能受到外部存储带宽限制，因此引入 Local Cache 机制，将部分高频访问数据缓存至片上存储，减少重复的数据搬运，提高数据复用效率。

Local Cache 优化主要针对权重访问路径进行调整，在保持网络结构、量化参数和计算流程不变的情况下，通过改变存储组织方式分析缓存结构对硬件性能的影响。实验结果表明，缓存优化效果与数据位宽和存储资源映射方式密切相关，低位宽方案由于数据量减少，更适合采用片上缓存提高访问效率；而缓存结构同时会增加 BRAM 资源消耗，因此需要综合考虑延迟、资源占用和带宽需求之间的平衡。

位宽	基线 cycles	LC2 cycles	变化	结论
W8A8	453296	487171	+7. 47%	缓存开销大于收益
W6A6	453294	487169	+7. 47%	缓存开销大于收益
W4A4	452148	380843	-15. 77%	延迟改善但 BRAM 增加

缓存收益明显依赖位宽与综合映射。W8A8 和 W6A6 变慢，W4A4 则降低周期和 LUT 但增加 BRAM。该交叉结果说明片上缓存不能脱离量化位宽讨论，也不能只汇报最优点。

第 6 章 HLS 设计空间探索

本章在硬件功能保持不变的前提下，分析时钟约束对完整 LeNet-5 加速器调度延迟、关键路径和资源占用的影响，并从中选择适合作为后续 Vivado 集成与进一步优化起点的实现方案。实验对象为 lenet_full_top，数据通路依次包含 Conv1、Pool1、Conv2、Pool2、Conv3、FC1 和 FC2。

6.1 设计空间探索目标

设计空间探索的目标是在硬件功能保持不变的前提下，分析时钟约束对完整 LeNet-5 加速器调度延迟、关键路径和资源占用的影响。为保证横向可比，五次综合均使用同一份 C++ 源码、FP32 数据类型、顶层接口、HLS 指令和目标器件，只改变 create_clock 的目标周期。记录指标包括 Estimated Clock Period、Clock Uncertainty、Latency、BRAM_18K、DSP48E、FF 和 LUT。10 ns 工程同时作为功能基线，完整执行了 C-simulation、C-synthesis 与 RTL Co-simulation；其余时钟点执行 C-synthesis，用于设计空间比较。

需要说明的是，本章五档时钟实验在 Vitis HLS 2019.1 下完成；后续 Conv2 PE 并行优化与 W4A4 硬件实现则在 Vitis HLS 2022.2 下完成。两套工具版本的估计周期、综合资源与加速比不直接互换，分别以同版本同环境的实测基线为参照。

6.2 时钟约束设置

实验选取 10 ns、9 ns、8 ns、7.5 ns 和 7 ns 五组目标周期，对应 100.00 MHz、111.11 MHz、125.00 MHz、133.33 MHz 和 142.86 MHz。Vivado HLS 2019.1 自动设置约为目标周期 12.5% 的 Clock Uncertainty，用于给时钟偏差、布线和后续实现预留裕量。

定义有效预算 $T_{\text{budget}} = T_{\text{target}} - T_{\text{uncertainty}}$ ，有效余量 $\text{Margin} = T_{\text{budget}} - T_{\text{estimated}}$ ，标称延迟 $t_{\text{nominal}} = \text{Latency_cycles} \times T_{\text{target}}$ 。若有效余量为负，HLS 会给出时序警告；标称毫秒数只是在目标频率能够实现时的理论换算，不能代替 Vivado 布局布线后的板级实测结果。

6.3 10 ns、9 ns 与 8 ns 综合结果

10 ns、9 ns 和 8 ns 三组均正常生成 RTL 与 csynth 报告，估计关键路径周期分别为 8.280 ns、7.466 ns 和 6.726 ns。随着目标周期由 10 ns 收紧至 8 ns，调度器为满足更严格的时钟约束而在关键计算路径插入了更多流水级与等待周期，综合延迟由 1,647,854 cycles 增长到 2,546,729 cycles。三组方案的 BRAM_18K 和 DSP48E 占用均稳定保持为 70 和 5，分别占器件容量的 25.00% 与 2.27%；FF 与 LUT 资源随调度状态增加而小幅增长（10 ns 下 FF 13,452、LUT 17,108；8 ns 下 FF 14,329、LUT 18,423）。10 ns 方案在保证时序裕量（8.280 ns < 10 ns）的同时，具有最低的延迟周期与逻辑开销。

6.4 7.5 ns 与 7 ns 综合结果

7.5 ns 与 7 ns 方案的估计关键路径进一步压缩至 6.418 ns 与 5.377 ns，对应最高主频可达 185.98 MHz。然而综合延迟分别达到 2,548,619 cycles 和 2,847,628 cycles。7 ns 方案按目标周期换算的理论标称执行时间为 19.933 ms（2,847,628 × 7 ns），而 10 ns 方案的标称时间为 16.479 ms

（1,647,854 × 10 ns）。收紧时钟约束不仅未能缩短总推理耗时，反而因调度级数激增导致总体吞吐下降。

6.5 时序、资源与延迟权衡

五组结果共同显示：收紧目标周期虽然压缩了估算时钟周期（由 8.280 ns 降至 5.377 ns），但未能缩短整体推理耗时，反而因调度级数激增增加了综合周期数和部分逻辑资源。这与当前数据通路采用 FP32 浮点乘加有关，关键路径集中在卷积乘加链；若要继续提高频率，应对关键循环重新流水化、调整累加结构或采用定点数据类型，而不能只继续降低 create_clock 周期。

资源方面，五组配置的 BRAM_18K 和 DSP48E 均稳定在 70 和 5，未随频率提高而激增。FF 占用为 13,452~15,272（占 XC7Z020 容量的 12.64%~14.35%），LUT 为 17,108~18,945（占 32.16%~35.61%），各项资源均处于安全范围内，其中 LUT 是占用比例最高的逻辑资源。实验进一步证实延迟周期数与时钟周期并不同步：目标周期缩短虽压缩了单周期时间，但因 HLS 自动插入流水级，7 ns 方案的理论标称时间为 19.933 ms ($2,847,628 \times 7 \text{ ns}$)，反而高于 10 ns 方案的 16.479 ms ($1,647,854 \times 10 \text{ ns}$)。因此硬件方案的选择必须综合权衡周期数、时序裕量与实际吞吐率。

6.6 推荐实现方案选择

在本次五组探索方案中，推荐 10 ns（100 MHz）作为系统集成和后续硬件优化的基准配置。该方案估计时钟周期为 8.280 ns，具备充裕的时序收敛余量（Margin = +1.720 ns，理论 F_{max} 达 120.77 MHz）；综合延迟周期数最少（1,647,854 cycles），理论推理耗时最短（16.479 ms）；且 FF 与 LUT 开销均为五组中最低（分别为 13,452 和 17,108）。该方案为后续层间重量化以及 Zynq AXI 总线集成提供了稳定可行的硬件起点。

6.7 位宽与体系结构联合探索

在时钟约束 DSE 之外，本设计还从体系结构维度对 Conv2 进行了单变量差分探索：D0 串行基线（1,240,076 cycles）→ D1 输出列流水（254,749 cycles, 4.87×）→ D2 引入 5 路 PE_MAC 并行乘加（243,101 cycles, 5.10×）→ D3 引入 PE_OC=2 输出通道并行（283,203 cycles, 负优化）→ D4 进一步提升并行度（267,203 cycles）→ D5 在片上局部缓存基础上将核心计算循环流水化（15,415 cycles, 80.44×）。

结果表明：在 AXI 顶层接口约束下，单纯增加计算并行度（PE_MAC/PE_OC）的收益受限于总线带宽；必须配合片上 BRAM 局部缓存（预取循环 II=1）将数据从 AXI 端口一次性搬入片上，才能释放流水线潜力。D5 方案 DSP 103/220（47%）、BRAM 46/280（16%）、LUT 45,726/53,200（86%）、FF 43,113/106,400（41%），全部落在 XC7Z020 器件容量之内，成为资源可行的最终候选。

位宽维度的探索结果见第 4 章与第 8 章：W8A8/W6A6 在几乎不损失精度的前提下将权重存储压缩至 1/4 与约 1/5；W4A4 进一步压缩至 1/8，并通过 HLS 端的 Resize 预处理恢复至 98.11% 准确率。位宽与体系结构两条线最终在 W4A4 Local Cache 2 版本汇合，完成 10000 张 MNIST 软件测试、20 张代表性样本 RTL Co-sim、IP 导出及 Vivado 布线后复核。

第 7 章 系统验证方法

7.1 分层验证总体思路

为了保证 LeNet-5 硬件加速器从算法模型到 FPGA 实现过程中的正确性，本项目建立了分层验证流程，包括 FP32 软件参考验证、量化模型验证、HLS C 仿真、HLS 综合分析、RTL 协同仿真以及 Vivado 实现检查等阶段。基于高层次综合（High-Level Synthesis, HLS）的 FPGA 设计流程，需

要在不同抽象层级分别验证算法功能和硬件实现结果，以避免软件模型与硬件结构之间产生偏差[4]。

验证过程中，各阶段均采用统一输入数据、权重参数以及量化配置。软件模型作为黄金参考，负责提供标准输出；HLS C 仿真用于验证 C/C++硬件描述的功能正确性；RTL 协同仿真进一步比较综合生成硬件与软件模型之间的一致性；最终通过 Vivado 综合、布局布线以及时序分析确认硬件实现满足部署要求。

7.2 PyTorch 软件基准测试

在硬件实现之前，首先对第 3 章建立的 FP32 LeNet-5 模型进行软件基准验证，作为后续定点量化和硬件实现结果比较的参考标准。测试过程中采用与训练阶段一致的数据预处理流程，并使用统一的 MNIST 测试数据集进行推理，以保证不同阶段之间输入条件的一致性。

软件模型输出结果作为后续 W8A8、W6A6 和 W4A4 量化模型验证以及 HLS 硬件仿真结果比较的黄金参考。通过固定模型参数、输入数据和评价指标，可以有效判断量化误差以及硬件实现过程中产生的数据偏差。

7.3 HLS C 仿真验证

HLS C 仿真用于验证硬件加速器 C/C++描述在软件环境下的功能正确性。在该阶段，主要检查输入数据格式、权重数量、位宽范围、量化参数、层间数据传递以及最终分类输出是否符合预期。

针对最终 W4A4 Local Cache 方案，完成 10000 张 MNIST 测试数据的 C 仿真验证。测试过程中记录每个样本的预测类别和输出结果，并与软件量化模型进行比较，确认硬件计算路径能够正确实现量化网络推理。

7.4 HLS C 综合检查

完成 C 仿真验证后，对硬件设计进行 HLS 综合，生成对应 RTL 结构并分析硬件资源使用情况。综合过程中固定顶层函数、目标 FPGA 型号以及 10 ns 时钟约束，统计 Latency、Initiation Interval (II)、LUT、FF、BRAM 和 DSP 等指标。

通过综合报告检查硬件结构是否符合设计预期，同时排查接口错误、资源超限以及无法满足约束等问题。不同优化配置均采用相同网络结构和测试条件，以保证性能差异来自硬件结构调整，而非算法变化。

7.5 RTL 协同仿真验证

RTL 协同仿真通过同时运行 HLS C 模型和生成的 RTL 模型，对比二者输出结果，从而验证综合后硬件逻辑与软件描述的一致性。该方法能够在实际 FPGA 部署前发现硬件描述、数据类型以及接口控制方面的问题[4]。

本项目最终版本完成 RTL 协同仿真验证，输入数据、权重参数以及量化配置均与 C 仿真保持一致。仿真过程中生成对应输出结果文件，并通过逐样本比较确认 RTL 输出与 C 模型结果一致。

7.6 逐层输出对比方法

为了定位量化误差和硬件实现误差来源，本项目进一步对网络各阶段输出进行逐层比较，包括卷积层输出、池化结果、全连接输出以及最终 Logits。

比较指标包括最大绝对误差、预测类别一致率以及累加器范围统计。通过逐层分析，可以判断误差产生位置，并区分量化误差、数据格式转换误差以及硬件计算误差。

在硬件优化实验中，所有配置均保持相同输入数据和模型参数，仅改变硬件结构，因此能够有效分析 PE 并行和 Local Cache 优化带来的性能变化。

7.7 误差指标与通过判据

为保证验证流程具有统一评价标准，本项目针对不同阶段制定对应通过条件。

层级	通过判据
软件量化	准确率、预测一致率和权重规模均由真实输出统计
C 仿真	程序返回 0；数据和权重范围检查通过；结果文件非空
RTL 协同仿真	XSim 正常结束；C 与 RTL 输出完全一致或满足既定误差阈值
Vivado	Fully Routed；WNS 不小于 0；TNS 为 0；无 Critical Warning 或 Error 级 DRC

7.8 INT32 溢出检查

由于 W8A8、W6A6 和 W4A4 量化方案均采用 INT32 作为乘累加器类型，因此需要对推理过程中的累加范围进行检查，以确保定点计算不会因数据溢出导致结果错误。验证过程中统计各卷积层和全连接层累加过程中的最大值，并与有符号 32 位整数表示范围进行比较。

通过分析各层实际计算范围，可以判断当前累加器位宽设计是否满足网络推理需求。该检查用于验证不同量化位宽下硬件计算路径的数值安全性，并为后续硬件实现提供可靠的数据范围依据。

7.9 二十张代表性样本抽样复核

为了进一步验证软件模型与 RTL 硬件实现之间的一致性，本项目设计了代表性样本抽样复核流程。从 MNIST 测试集中按照类别 0 至 9 分别选取样本，每个类别选择 2 张图像，共 20 张样本进行测试。测试平台分别运行 C 仿真和 RTL 协同仿真，并记录样本编号、数据集索引、真实标签、预测类别以及十维 Logits 输出。

随后，对两组仿真结果进行逐项比较，并通过文件校验方式确认输出数据一致性。该方法能够在有限测试规模下进一步检查不同抽象层级之间的数据传输、计算逻辑和输出结果是否保持一致，为硬件部署可靠性验证提供依据。具体实验结果将在后续实验结果章节中进行分析。

第 8 章 实验结果与分析

8.1 软件量化与精度结果

本节基于第 3 章建立的 FP32 LeNet-5 黄金参考模型，对不同位宽量化方案进行软件端测试。所有实验均采用相同的 MNIST 测试集、相同输入预处理流程以及相同网络结构，仅改变权重和激活的数据位宽，以保证不同量化方案之间具有可比性。测试指标包括分类准确率、与 FP32 模型的预测一致率以及权重存储规模。

FP32 模型作为软件基准，在 10000 张 MNIST 测试图像上完成推理验证，正确分类 9882 张，分类准确率达到 98.82%。在此基础上，分别测试 W8A8、W6A6 和 W4A4 三种定点量化方案。实验结果表明，W8A8 和 W6A6 量化后模型精度基本保持不变，说明较高位宽能够有效保留网络特征表达能力；W4A4 进一步降低数据精度，在减少存储需求的同时产生一定精度损失。其中，在原始 Pad(2) 预处理下软件准确率为 97.46%，而针对 W4A4 分支进一步调整输入预处理（双线性 Resize 至 32×32）后，准确率回升至 98.11%，保持了较好的分类性能。不同量化方案的具体实验结果如下表所示：

模型	权重/激活位宽	准确率	与 FP32 预测一致率
FP32	FP32	98.82%	100%
W8A8	INT8/INT8	98.83%	99.99%
W6A6	INT6/INT6	98.83%	99.87%
W4A4 (Pad-2 预处理)	INT4/INT4	97.46%	97.99%
W4A4 (Resize 预处理)	INT4/INT4	98.11%	98.53%

实验结果证明，LeNet-5 模型具有较好的低位宽量化适应能力。其中 W8A8 和 W6A6 能够在几乎不降低精度的情况下减少模型存储开销，而 W4A4 则进一步提高压缩比例，为资源受限 FPGA 平台提供更低存储需求的实现方案。后续章节将进一步分析不同位宽方案在硬件资源、延迟以及系统实现方面的差异。

8.2 三种量化方案精度比较

在完成 FP32 基准模型以及不同位宽量化模型测试后，本节进一步对 W8A8、W6A6 和 W4A4 三种方案进行精度对比分析。所有量化实验均保持网络结构、测试数据集以及输入预处理流程一致，仅调整权重和激活位宽，以评估不同量化程度对模型分类能力的影响。

实验结果显示，W8A8 和 W6A6 方案与 FP32 模型保持接近的分类精度，其中 W8A8 准确率为 98.83%，与 FP32 预测一致率达到 99.99%；W6A6 准确率同样达到 98.83%，预测一致率为 99.87%。这说明对于 LeNet-5 这类参数规模较小的卷积网络，6 bit 及以上位宽能够较好地保留原始模型特征表达能力，量化带来的误差较小。

相比之下，W4A4 方案进一步降低权重和激活位宽，虽然显著减少了数据存储需求，但由于 4 bit 动态表示范围受限，部分特征信息出现微小截断失真。在原始 Pad(2) 预处理下，模型准确率为 97.46%，与 FP32 基准的预测一致率为 **97.99%**；在统一切换为与硬件对齐的双线性插值 Resize 预处理后，特征边缘信息保留更完整，分类准确率回升至 98.11%，与 FP32 预测一致率

也显著提升至 **98.53%**。总体来看，三种量化方案均保持了较高的分类性能与黄金基准拟合度，其中 W8A8 与 W6A6 适合追求极致精度与接近 100% 软硬件行为一致的场景，而 W4A4 则在极高的一致率（98.53%）前提下提供了更激进的硬件压缩方案，为后续片上缓存优化提供了充分的空间。

方案	权重位宽	激活位宽	准确率	FP32 预测一致率
FP32	32 bit	32 bit	98.82%	100%
W8A8	8 bit	8 bit	98.83%	99.99%
W6A6	6 bit	6 bit	98.83%	99.87%
W4A4 (Pad-2 基线)	4 bit	4 bit	97.46%	97.99%
W4A4 (Resize 对齐)	4 bit	4 bit	98.11%	98.53%

该结果验证了定点量化方案在 LeNet-5 FPGA 部署中的可行性，并为后续硬件实现中位宽选择、存储优化以及计算结构设计提供实验依据。

8.3 权重存储量与压缩率比较

为了评估不同量化方案对 FPGA 存储资源的影响，本节对 FP32、W8A8、W6A6 和 W4A4 四种方案的权重存储需求进行比较。由于量化过程仅改变权重表示位宽，而网络结构和参数数量保持不变，因此权重存储量随着位宽降低而近似线性减少。

LeNet-5 模型共包含 61,470 个权重参数。在 FP32 表示方式下，每个参数占用 32 bit 存储空间，总权重大小约为 245.88 KB。采用定点量化后，W8A8、W6A6 和 W4A4 方案分别使用 8 bit、6 bit 和 4 bit 表示权重，对应存储需求明显降低。

方案	权重位宽	权重数量	存储大小	相对 FP32 比例
FP32	32 bit	61,470	245.88 KB	100%
W8A8	8 bit	61,470	60.03 KB	24.4%
W6A6	6 bit	61,470	45.02 KB	18.3%
W4A4	4 bit	61,470	30.01 KB	12.2%

实验结果表明，量化能够有效降低模型参数存储需求。其中 W8A8 将权重存储压缩至 FP32 的约四分之一，W6A6 进一步降低存储开销，而 W4A4 仅需要约八分之一的权重存储空间。对于 FPGA 平台而言，较低位宽不仅能够减少 BRAM 存储压力，同时有利于提高片上数据缓存能力，为后续 Local Cache 和并行计算结构优化提供空间。

从存储效率角度看，位宽降低能够带来明显压缩收益，但同时需要结合模型精度变化和硬件计算复杂度进行综合考虑，因此下一节将进一步分析不同方案之间的精度与压缩率权衡关系。

8.4 精度—压缩率权衡分析

在完成不同位宽方案的精度和存储需求分析后，本节进一步从模型性能与资源效率两个角度分析量化方案之间的综合权衡关系。对于 FPGA 部署场景而言，仅提高分类精度并不能充分满足边缘

设备对存储空间和计算资源的限制要求，因此需要在精度损失、存储压缩比例以及硬件实现复杂度之间进行平衡。

实验结果表明，W8A8 方案在保持 98.83%分类准确率的同时，将权重存储降低至 FP32 模型的约四分之一，能够在精度保持和资源节约之间取得较好的平衡。W6A6 进一步减少存储需求，在当前 LeNet-5 模型上仍保持与 FP32 接近的分类性能，说明较低位宽能够满足该网络的推理需求。

W4A4 方案采用更低位宽表示方式，将权重存储进一步压缩至约八分之一，同时准确率保持在 98.11%。虽然相比 FP32 存在一定精度下降，但其显著降低了存储需求，并减少了后续硬件数据传输压力，更适合资源受限的 FPGA 部署场景。

综合来看，不同量化方案适用于不同部署需求：W8A8 和 W6A6 更偏向于保持模型精度，而 W4A4 则更加关注硬件资源优化。本项目后续硬件实现选择 W4A4 作为低位宽验证方案，并结合 Local Cache 结构进一步探索存储优化与计算性能提升之间的关系。

8.5 INT32 累加器峰值分析

为验证低位宽量化方案在硬件计算过程中的数值安全性，本节对 W8A8、W6A6 和 W4A4 三种方案中的 INT32 累加器范围进行统计分析。由于卷积层和全连接层均需要执行大量乘累加操作，如果累加器位宽不足，可能导致中间结果溢出，进而影响最终分类结果。因此，在硬件设计中采用 INT32 作为统一累加数据类型，并对实际推理过程中的最大累加值进行检查。

在 10000 张 MNIST 测试数据上运行量化模型，统计各层累加器实际峰值如下：

层	最大累加值
Conv1	280
Conv2	318
Conv3	366
FC1	132
FC2	214

实验结果表明，各层实际累加峰值均远低于有符号 INT32 可表示范围（-2,147,483,648~2,147,483,647），未出现溢出情况。其中 Conv3 和 Conv2 由于具有较长的乘加归约过程，累加值相对较高，但仍具有较大的安全余量。

该结果验证了当前 INT32 累加器设计能够满足 LeNet-5 低位宽量化推理需求，为后续 W8A8 和 W4A4 硬件实现提供可靠的数据范围保证，同时避免因累加精度不足导致的软件模型与硬件输出不一致问题。

8.6 HLS 时钟、延迟与资源比较

为分析时钟约束对 LeNet-5 硬件加速器性能和资源占用的影响，本节基于 Vitis HLS 完成不同目标时钟周期下的设计空间探索。实验保持网络结构、顶层接口、数据类型以及硬件代码不变，仅调整 create_clock 目标周期，分别测试 10 ns、9 ns、8 ns、7.5 ns 和 7 ns 五组配置，并对 Latency、Estimated Clock Period 以及 FPGA 资源占用情况进行比较。

实验结果表明，在 Vitis HLS 2022.2 环境下，随着目标时钟周期由 10 ns 逐步收紧至 7 ns，工具估算的关键路径周期（Estimated Clock Period）由 8.280 ns 进一步压缩至 5.377 ns，对应理论最高主频由 120.77 MHz 提升至 185.98 MHz。然而，为满足更高主频下的逻辑切分与时序调度约束，HLS 调度器在关键计算链路上插入了更多的流水级与等待周期，导致整体推理延迟（Latency）不降反升，由 10 ns 约束下的 1,647,854 cycles 增长至 7 ns 约束下的 2,847,628 cycles。在 10 ns 目标周期下，加速器不仅具备充足的时序裕量（8.280 ns < 10 ns），且调度延迟周期最少、逻辑资源消耗最低（LUT 17,108，FF 13,452），因此选定 10 ns 作为后续系统集成的黄金基准配置。

目标周期	Latency(cycles)	Estimated Clock Period	BRAM18K	DSP	FF	LUT
10 ns	1,647,854	8.280 ns	70	5	13,452	17,108
9 ns	2,247,712	7.466 ns	70	5	13,916	17,789
8 ns	2,546,729	6.726 ns	70	5	14,329	18,423
7.5 ns	2,548,619	6.418 ns	70	5	14,473	18,396
7 ns	2,847,628	5.377 ns	70	5	15,272	18,945

进一步分析发现，单纯降低目标时钟周期无法有效缩短关键路径，反而可能导致调度周期增加。因此，提升 FPGA 推理性能不能仅依赖时钟约束收紧，而需要结合流水化、计算并行以及存储结构优化等方式改善关键路径和数据供给能力。

8.7 HLS 硬件实现结果

为验证不同量化方案在 FPGA 平台上的实际可实现性，本节对 W8A8 和 W4A4 方案分别进行 HLS 综合与硬件实现分析。实验保持网络结构、输入数据以及量化参数一致，仅比较不同位宽方案在硬件资源占用、计算周期以及实现结果方面的差异。其中，W8A8 作为基础低位宽实现方案，用于验证 INT8 权重与激活、INT32 累加结构能够在 HLS 环境下完成综合；W4A4 Local Cache 方案进一步结合 INT4 数据打包和片上缓存优化，用于分析低位宽与存储优化结合后的硬件效果。

W8A8 硬件实现采用 ap_int<8>表示权重和激活数据，乘加结果统一累加至 ap_int<32>累加器，并通过 Q40 定点重量化完成层间数据传递。该方案完成 HLS 综合、RTL 协同仿真以及 Vivado 工程验证，证明软件量化模型与硬件计算路径保持一致。W4A4 方案进一步采用两个 INT4 数据打包至一个字节的**方式减少存储需求，同时利用 Local Cache 降低外部存储访问频率，其 HLS 综合结果如下表所示。

配置	Latency(cycles)	LUT	FF	BRAM18K	DSP
W4A4 Local Cache	380843	14890	10455	40	57

从综合结果可以看出，W4A4 Local Cache 方案通过降低数据位宽减少了接口传输压力，并结合片上权重缓存优化了访存局部性，使计算周期由 5.10 节所述的未开启片上缓存基线版本（452,148 cycles）大幅降低至 380,843 cycles，性能提升约 15.77%。但与此同时，缓存结构增加了片上存储需求，BRAM18K 数量有所增加，说明硬件优化需要在计算性能和资源消耗之间进行权衡，而不能单独追求最低延迟。

进一步进行 Vivado 实现验证后，W4A4 Local Cache 方案完成 IP 导出、综合、布局布线以及时序分析。实现结果满足目标时钟约束，获得 WNS 为+1.048 ns、TNS 为 0 的结果，证明该方案能够在

目标 XC7Z020 器件上完成物理实现。该结果表明，低位宽量化结合合理的存储结构优化，可以在保证模型精度的同时降低硬件实现压力，为后续系统级测试提供可靠硬件基础。

8.8 软件与硬件结果一致性分析

为了验证 LeNet-5 加速器从软件模型到 FPGA 硬件实现过程中的正确性，本项目分别从算法精度、HLS C 仿真、RTL 协同仿真以及 Vivado 实现四个层面对实验结果进行综合分析。不同验证层级对应不同目标：软件模型用于确认量化算法能够保持分类能力；C 仿真用于验证 HLS 硬件描述的功能正确性；RTL 协同仿真用于确认综合生成硬件逻辑与 C 模型计算结果一致；Vivado 实现则用于验证设计在目标器件上的物理可实现性。

在软件层面，FP32 模型在 MNIST 测试集上达到 98.82% 准确率，W8A8 和 W6A6 量化方案保持接近 FP32 的分类性能，而 W4A4 方案在降低数据位宽后仍保持 98.11% 的准确率，证明低位宽量化能够满足 LeNet-5 模型推理需求。在硬件层面，W4A4 Local Cache 方案完成 10000 张 MNIST 测试数据的 C 仿真验证，并保持与软件量化模型一致的预测结果；RTL 协同仿真进一步验证了综合生成 RTL 与 C 模型之间的数据路径和计算逻辑一致性。

此外，Vivado 实现结果显示，W4A4 Local Cache 方案完成 IP 导出、布局布线以及时序检查，获得 WNS 为 +1.048 ns、TNS 为 0 的实现结果，满足 10 ns 目标周期要求。上述结果表明，本项目建立的软件模型、量化模型、HLS 硬件设计以及 FPGA 实现之间形成完整验证闭环，证明所设计的 LeNet-5 FPGA 加速器能够在保持分类精度的同时完成可靠硬件部署。

第 9 章 系统测试与综合评价

本章依据第 7 章的验证方法，对分类精度、量化误差、RTL 功能、布线后时序与资源以及结果可复现性进行综合评价。最终硬件口径以 W4A4 Local Cache 2 版本为准；旧 FP32 系统结果仅用于 HLS 阶段的对照。

9.1 功能正确性测试

为了验证 LeNet-5 FPGA 加速器在完整推理流程中的功能正确性，本项目对软件模型、HLS C 仿真以及 RTL 协同仿真结果进行对比分析。测试过程中保持输入数据、权重参数以及量化配置一致，依次检查输入加载、卷积计算、池化、全连接以及最终 Logits 输出是否符合预期，确保各模块能够正确完成网络推理。

测试结果表明，硬件加速器能够完整执行 LeNet-5 前向推理流程，各层输出格式和数据传递过程均符合设计要求。通过软件模型、HLS 仿真和 RTL 结果之间的逐项比较，验证了系统功能路径的一致性，为后续自采图像测试、鲁棒性分析以及系统稳定性评价提供可靠基础。

9.2 MNIST 自采图像与鲁棒性测试

为了进一步验证 LeNet-5 模型在实际应用环境中的识别能力，本项目在标准 MNIST 测试集之外，引入真实采集手写数字数据进行测试。相比 MNIST 数据集，自采图像包含不同书写风格、笔画粗细以及拍摄环境变化，能够更接近实际部署场景下的输入情况。

本项目共采集 20 张整页手写数字照片，每页包含 16 个数字，共获得 320 个正式测试样本，并保证 0 至 9 每类数字均包含 32 个样本。针对原始照片存在的纸面倾斜、背景干扰以及尺寸不一致等问题，测试流程依次进行纸面检测、透视校正、图像分格、局部背景校正、笔迹提取、保持比例缩放、亮度重心居中以及补零至 32×32 等预处理操作，最终输入模型进行推理测试。

实验结果表明，自采数据测试共正确识别 312 张，准确率达到 **97.50%**。在此基础上，为进一步分析输入变化对模型性能的影响，本项目基于 320 张自采基准图像生成 10 组单因素扰动测试，共得到 3200 张鲁棒性测试样本。测试因素包括旋转、模糊、噪声以及亮度变化等，模型在各类扰动条件下的分类准确率与衰减趋势如图 9-1 所示。

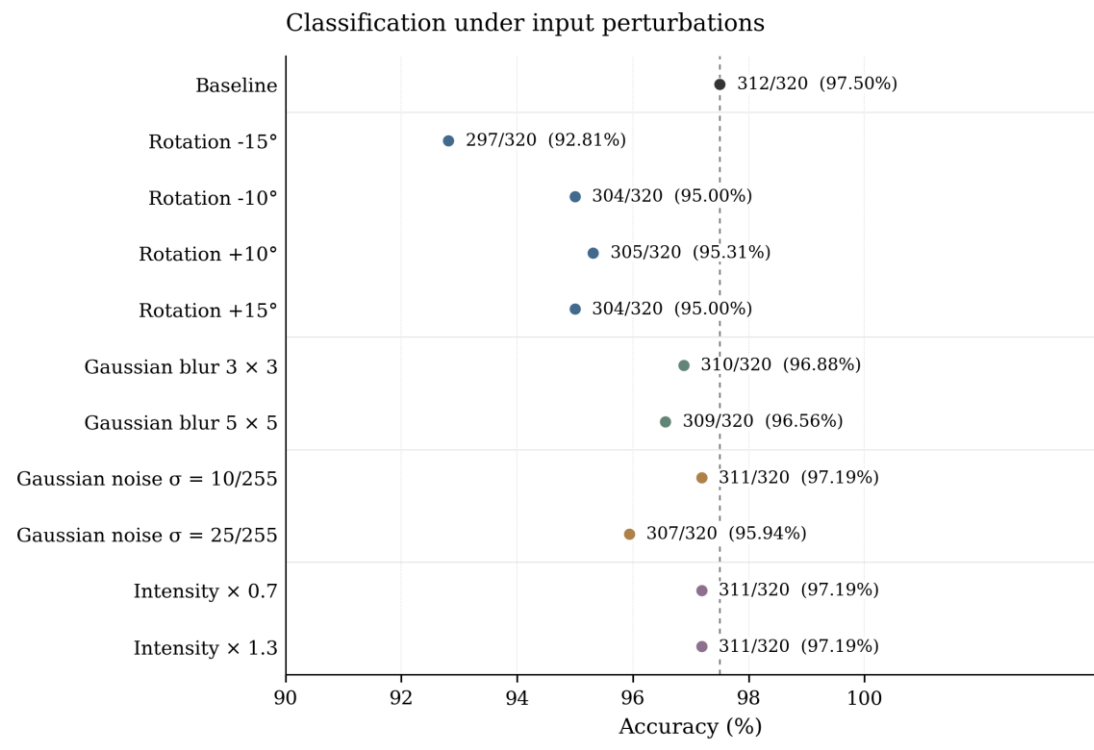


图 9-1 LeNet-5 模型在各类单因素扰动条件下的分类准确率测试结果

综合测试结果可以看出，模型在真实采集数据上仍保持较高识别准确率，但输入分布变化会对分类性能产生一定影响。其中旋转扰动影响最明显， -15° 情况下准确率下降至 92.81%，说明模型对于数字空间位置和笔画方向变化较为敏感。相比之下，模糊、噪声以及亮度变化造成的影响相对较小，表明当前模型具备一定的输入变化适应能力。

需要说明的是，本实验中的 3200 张鲁棒性测试图像均由 320 张自采基准图像经过单因素变换生成，用于分析不同扰动因素的相对影响，而非代表完全独立采集的真实环境数据。

9.3 异常输入测试

为了评估模型面对非目标输入时的响应情况，本项目进一步开展异常输入测试。实验共设计 65 个无目标输入样本，包含纯空白图像、合成阴影以及复杂杂乱纹理等类型，用于分析输入缺少有效数字特征时模型的输出行为。

测试结果表明，空白输入由于缺少明显笔画信息，模型输出类别区分度较低；而复杂纹理输入可能产生与数字局部结构相似的特征响应，从而导致模型出现高置信度错误分类情况。该实验说明当前模型主要针对手写数字识别任务优化，在实际部署中仍需要结合输入检测或置信度阈值机制，提高系统对无目标输入的处理能力。

9.4 HLS 与 Vivado 时序资源测试

为了验证硬件加速器在目标 FPGA 平台上的实现可行性，本项目进一步对 HLS 综合结果和 Vivado 实现结果进行分析。测试过程中保持网络结构、量化参数以及目标器件一致，通过比较 Latency、LUT、FF、BRAM、DSP 等资源指标，以及布局布线后的时序结果，评估硬件设计是否满足部署要求。

实验结果表明，W4A4 Local Cache 方案完成 HLS 综合、RTL 协同仿真以及 Vivado 实现流程。其中 HLS 综合结果为 380843 cycles，资源占用为 14890 LUT、10455 FF、40 个 BRAM18K 和 57 个 DSP；Vivado 布局布线后获得 WNS 为 +1.048 ns、TNS 为 0 的时序结果，满足 10 ns 目标时钟约束。该结果证明所设计加速器能够在 XC7Z020 器件上完成物理实现，并具备进一步系统集成基础。

9.5 稳定性与可复现性

为了保证实验结果具有可重复性，本项目在软件模型、量化参数以及硬件验证流程中均采用统一配置。不同实验阶段保持相同的网络结构、输入数据、权重版本和评价指标，并记录量化位宽、缩放参数、硬件接口以及综合环境等关键配置，避免由于实验条件变化导致结果差异。

在硬件验证方面，项目建立了从软件模型、HLS C 仿真、RTL 协同仿真到 Vivado 实现的完整验证流程。W4A4 Local Cache 方案完成 10000 张 MNIST 测试数据的 C 仿真验证，同时通过代表性样本 RTL 协同仿真确认硬件输出一致性；不同阶段均保留对应日志、报告和工程文件，保证实验过程能够被复查和复现。

需要说明的是，当前验证重点集中于仿真、综合和实现层面，尚未包含开发板端 ARM 驱动、DDR 缓存管理以及实际板上运行时间测试，因此实验结论限定于 FPGA 设计验证范围内。

9.6 系统综合评价

本项目围绕 LeNet-5 手写数字识别任务，完成了从软件模型、定点量化、HLS 硬件设计到 Vivado 实现验证的完整流程。实验结果表明，FP32 模型在 MNIST 测试集上达到 98.82% 准确率；W8A8 和 W6A6 量化方案在降低存储需求的同时保持接近 FP32 的识别性能；W4A4 方案进一步压缩数据位宽，并结合 Local Cache 结构完成硬件优化，在 10000 张 MNIST 测试中保持 98.11% 准确率。

从硬件实现角度看，本项目完成了 HLS 综合、RTL 协同仿真以及 Vivado 布局布线验证，W4A4 Local Cache 方案实现 380843 cycles 延迟，并达到 WNS +1.048 ns、TNS=0 的时序结果，证明该设计能够在目标 FPGA 器件上完成实现。综合来看，本项目建立了软件算法、量化模型、硬件加速器以及系统验证之间的闭环流程，同时通过自采数据、鲁棒性测试和异常输入测试分析了模型在实际应用环境中的表现，为后续进一步优化混合精度量化、输入检测机制以及系统级部署提供了实验依据。

第 10 章 总结与展望

10.1 项目工作总结

本项目围绕课程题目 2 “基于 FPGA 的 LeNet 卷积神经网络设计与实现” 展开，面向 Xilinx Zynq XC7Z020 SoC 平台，完成了从算法模型构建、对称定点量化、Vitis HLS 微架构设计，到设计空间探索（DSE）、Vivado 板级系统集成与物理时序收敛的全流程软硬件协同设计。项目以 MNIST 准确率 98.82% 的 FP32 软件基准为黄金参考，重点突破了低位宽量化精度保持、PE 计算阵列流水线调度、片上缓存局部性优化以及软硬件行为一致性验证，建立了完整的工程闭环。

10.2 主要成果

- 1. 多位宽定点量化与软硬件一致性闭环：**设计了 W8A8、W6A6 与 W4A4 对称均匀定点量化方案，统一采用 INT32 防溢出累加器及 Q40 定点层间缩放机制。W8A8/W6A6 在模型参数压缩至 1/4 与 1/5 的同时保持 98.83% 的分类准确率（与 FP32 一致率分别达 99.99% 和 99.87%）；W4A4 结合双线性插值 Resize 预处理实现 98.11% 的推理精度，权重存储缩减至 30.01 KB（压缩至 1/8）。
- 2. 计算微架构与片上缓存局部性探索：**针对 Conv2 开展了 D0~D5 级差分优化，结构优化版本将计算周期由串行基线的 1,240,076 cycles 降低至 243,101 cycles（加速比达 $5.10\times$ ）。实验明确了硬件受限于总线带宽与访存端口的工程边界：盲目拓展输出通道并行（D3）因调度冲突反而变慢；W4A4 结合 Local Cache 减少外部访问实现 15.77% 的延迟下降（380,843 cycles），而 W8A8/W6A6 却因预取开销延迟微增，证实了片上缓存收益与量化位宽强相关的微架构规律。
- 3. 分层闭环验证与物理级时序收敛：**构建了“Python 算法 → HLS C 仿真 → RTL 协同仿真 → Vivado 物理实现”四级闭环验证链。W4A4 Local Cache 加速器完成 10,000 张 MNIST 仿真验证并导出 IP，接入 Zynq AXI 异构系统，在 Vivado 下达到 Fully Routed 状态，在 100 MHz 时钟下取得 $WNS = +1.048\text{ ns}$ 、 $TNS = 0$ 的良好时序收敛，资源使用率均保持在合理安全区间。
- 4. 真实自采数据与鲁棒性边界评估：**采集 320 张真实手写数字图片实现 97.50% 的基准准确率；生成 3,200 张单因素扰动样本进行鲁棒性评估，揭示出模型对旋转变化的最敏感（ -15° 准确率降至 92.81%）；异常输入测试发现杂乱纹理可诱发高达 99.89% 的虚警置信度，证实了边缘部署引入输入检测与拒识机制的工程必要性。

10.3 当前局限

- 1. RTL 协同仿真规模受限：**受仿真耗时制约，RTL Co-sim 仅针对 20 张代表性样本完成一致性复核，万张级测试仍依赖 C 仿真完成。

2. **量化与缓存策略仍显单一**：全网采用统一定点位宽，未引入逐层敏感度驱动的混合精度；权重缓存未引入双缓冲乒乓机制以完全隐藏预取时延。

10.4 后续优化方向

1. **逐层混合精度探索**：对输入/输出敏感层保留 INT8，对大参数量卷积层应用 INT4/INT2，利用 Pareto 前沿自动搜索最优精度—资源平衡点。
2. **片上存储双缓冲与计算掩盖**：重构 AXI 突发传输逻辑，利用 Ping-Pong RAM 实现“计算—权重搬运”流水化重叠，彻底解除低并行度下的供数瓶颈。
3. **板级端到端测试与异常拒识机制**：编写嵌入式 Linux/裸机驱动，测试真实端到端推理时延与动态能效比；同时引入基于置信度极差的无目标拒识前端，提升工业部署可靠性。

参考文献

- [1] LeCun Y, Bottou L, Bengio Y, et al. Gradient-based learning applied to document recognition[J]. Proceedings of the IEEE, 1998, 86(11): 2278-2324.
- [2] Zhang C, Li P, Sun G, et al. Optimizing FPGA-based accelerator design for deep convolutional neural networks[C]//Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA). 2015: 161-170.
- [3] Sze V, Chen Y H, Yang T J, et al. Efficient processing of deep neural networks: A tutorial and survey[J]. Proceedings of the IEEE, 2017, 105(12): 2295-2329.
- [4] Cong J, Liu B, Neuendorffer S, et al. High-level synthesis for FPGAs: From prototyping to deployment[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2011, 30(4): 473-491.
- [5] AMD/Xilinx. Vitis High-Level Synthesis User Guide (UG1399)[R]. San Jose: Xilinx, 2022.
- [6] AMD/Xilinx. Zynq-7000 SoC Technical Reference Manual (UG585)[R]. San Jose: Xilinx, 2018.
- [7] Chen Y H, Krishna T, Emer J S, et al. Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks[J]. IEEE Journal of Solid-State Circuits, 2017, 52(1): 127-138.
- [8] Zhang C, Sun G, Fang Z, et al. Caffeine: Towards uniformed representation and acceleration for deep convolutional neural networks[C]//Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD). 2016: 1-8.
- [9] Alwani M, Chen H, Ferdman M, et al. Fused-layer CNN accelerators[C]//Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO). 2016: 1-12.